

CS 630: Software Testing and Quality Assurance

**4-2 Activity: API Interface Risk Analysis and Edge Case Testing**

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

July 29, 2026

## **Experience Identifying and Testing Edge Cases**

Completing this activity provides valuable hands-on experience with designing and executing API tests that extend beyond normal functionality. I began by reviewing the API implementation and the existing starter tests to understand how the /count endpoint processed requests and returned responses. The original project contained only two basic unit tests that verified the API correctly counted words for simple input. After confirming that these initial tests passed, I analyzed the API to identify additional edge cases that were not covered. Based on that analysis, I expanded the test suite by creating additional test cases for empty strings, whitespace-only input, very long strings, Unicode characters, missing query parameters, valid JSON requests, empty JSON objects, and null values. After updating the test file, I executed the test suite using pytest and successfully verified that all ten tests passed without errors. Achieving a successful result confirmed that the API behaved consistently across both standard and edge-case scenarios.

One of the most valuable aspects of this assignment was learning to think like a quality assurance engineer rather than only as a software developer. While developers often focus on implementing functionality that satisfies business requirements, QA engineers must also anticipate how applications behave when users provide unexpected or invalid input. This required me to carefully review the API documentation, examine the expected request and response formats, and identify areas where validation could potentially be weak. Designing additional test cases helped me consider situations that might not occur during normal use but could easily occur in production environments when different applications or users interact with the API.

This activity also reinforced the importance of defensive programming and comprehensive input validation. APIs act as communication points between multiple systems, making it impossible to assume that every request will contain valid or properly formatted data. Testing edge cases such as null values, missing parameters, empty JSON objects, and unsupported input formats helps verify that the application responds predictably rather than crashing or returning unexpected results. Similarly, testing long strings evaluates how the application handles larger workloads, while Unicode testing confirms that international characters are processed correctly. Together, these tests increase confidence that the API is capable of supporting a wider range of real-world usage scenarios.

Another important lesson I gained from this assignment was the relationship between software testing, reliability, and security. Many software defects and security vulnerabilities originate from insufficient input validation or improper handling of malformed requests. By intentionally creating tests that challenge the application's assumptions, quality assurance engineers can identify weaknesses before software is released into production. Even though this activity focused primarily on functionality, it demonstrated how systematic API testing contributes to reducing application failures, improving user experience, and strengthening the overall quality of a software system.

Successfully executing the complete test suite and seeing all ten tests pass demonstrated that the expanded test coverage effectively validated the API's behavior across multiple input conditions. Compared to the original two starter tests, the enhanced test suite provided significantly broader coverage and increased confidence in the reliability of the application. This experience highlighted that effective testing is not simply about verifying expected results but also about

identifying boundary conditions, validating error handling, and ensuring that software behaves consistently under unexpected circumstances.

To conclude, this activity strengthened both my technical testing skills and my understanding of software quality assurance. As a graduate student in Software Engineering, I found this assignment especially valuable because it combined theoretical concepts with practical implementation. Writing, executing, and validating API tests using pytest provided experience like professional QA workflows used in modern software development. Moving forward, I plan to continue incorporating edge-case analysis, boundary-value testing, and comprehensive input validation into my software development and testing process to create applications that are reliable, maintainable, secure, and capable of performing consistently in real-world environments.

```
(venv) malgorzatadebska@Malgorzatas-iMac 4-2 % pytest -v
===== test session starts =====
platform darwin -- Python 3.12.7, pytest-9.1.1, pluggy-1.6.0 -- /Users/malgorzatadebska/Desktop/4-2/venv/bin/python3.12
cachedir: .pytest_cache
rootdir: /Users/malgorzatadebska/Desktop/4-2
collected 10 items

test_word_count.py::test_get_single_word PASSED [ 10%]
test_word_count.py::test_get_two_words PASSED [ 20%]
test_word_count.py::test_empty_string PASSED [ 30%]
test_word_count.py::test_whitespace_only PASSED [ 40%]
test_word_count.py::test_long_string PASSED [ 50%]
test_word_count.py::test_unicode_text PASSED [ 60%]
test_word_count.py::test_missing_parameter PASSED [ 70%]
test_word_count.py::test_post_valid_json PASSED [ 80%]
test_word_count.py::test_post_empty_json PASSED [ 90%]
test_word_count.py::test_post_null_value PASSED [100%]

===== 10 passed in 0.09s =====
(venv) malgorzatadebska@Malgorzatas-iMac 4-2 %
```